D15A-04

ML-DL Exp-08

Design and Train a Convolutional Neural Network (CNN) on Fashion-MNIST Dataset

1. Dataset Source

Dataset: Fashion-MNIST
Provided by: Zalando Research
Source Link:
https://github.com/zalandoresearch/fashion-mnist
Note: In this experiment, the dataset is accessed using TensorFlow's built-in API, which automatically downloads it.

2. Dataset Description

Feature	Description
Total Samples	70,000
Training Data	60,000
Testing Data	10,000
Image Size	28 × 28 pixels
Channels	1 (Grayscale)
Number of Classes	10

Fashion-MNIST is a dataset of grayscale images representing fashion products.

Dataset Characteristics

Target Variable

- Multi-class label (0–9)
- Each number corresponds to a clothing category

Class Labels

Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

Input Features

- Pixel intensity values (0–255)
- Represent image patterns and textures

3. Mathematical Formulation of CNN

1. Convolution Operation Output feature map:

$$(I * K)(x, y) = \sum_i \sum_j I(x+i, y+j) \cdot K(i, j)$$

Where:

- I = Input image
- K = Kernel (filter)
- (x, y) = Pixel location

2. Activation Function (ReLU)

$$f(x) = \max(0, x)$$

3. Pooling (Max Pooling)

$$y = \max(x_1, x_2, ..., x_n)$$

4. Softmax Function (Output Layer)

$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

5. Loss Function (Cross-Entropy)

$$L = -\sum_i y_i \log(\hat{y}_i)$$

4. Algorithm Limitations

- Requires large dataset for better performance
- Computationally expensive (needs GPU for faster training)
- Prone to overfitting if model is too complex
- Difficult to interpret internal feature representations
- Sensitive to hyperparameter tuning
- Not suitable for very small datasets

5. Methodology / Workflow

Step-by-Step Process

6. Procedure

Step 1: Import Libraries

```
[1]
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
```

Step 2: Load Dataset

```
[2]
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.fashion_mnist.load_data()

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
```

Step 3: Explore Dataset

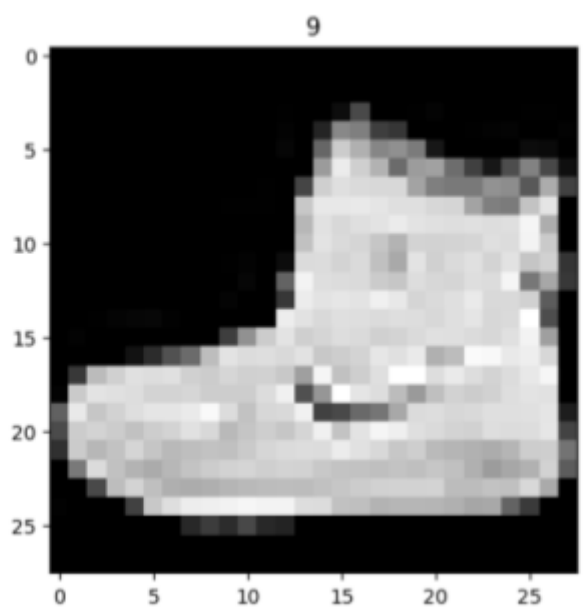
```
[3]
✓ 0s
print(train_images.shape)   # (60000, 28, 28)
print(test_images.shape)    # (10000, 28, 28)

...
(60000, 28, 28)
(10000, 28, 28)
```

Step 4: Data Preprocessing

```
[4]
✓ 0s
train_images = train_images / 255.0
test_images = test_images / 255.0
```

Step 5: Data Visualization

```
[5]
✓ 0s
plt.imshow(train_images[0], cmap='gray')
plt.title(train_labels[0])
plt.show()
```



Step 6: Build CNN Model

```
model = models.Sequential()

model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))
```

model.add(layers.Conv2D(64, (3,3), activation='relu'))

model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))

model.add(layers.Dense(10, activation='softmax'))

```
[4]
model = models.Sequential()

# Convolution Layer 1
model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(layers.MaxPooling2D((2,2)))

# Convolution Layer 2
model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))

# Convolution Layer 3
model.add(layers.Conv2D(64, (3,3), activation='relu'))

# Flatten + Dense
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))

# Output Layer
model.add(layers.Dense(10, activation='softmax'))

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` arg
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Step 7: Compile Model

```
[7]
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Step 8: Train Model

```
[8]
history = model.fit(train_images, train_labels,
                    epochs=10,
                    validation_data=(test_images, test_labels))

Epoch 1/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 62s 32ms/step - accuracy: 0.8167 - loss: 0.5038 - val_accuracy: 0.8662 - val_loss: 0.3715
Epoch 2/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 78s 30ms/step - accuracy: 0.8816 - loss: 0.3248 - val_accuracy: 0.8793 - val_loss: 0.3282
Epoch 3/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 57s 30ms/step - accuracy: 0.8984 - loss: 0.2762 - val_accuracy: 0.8926 - val_loss: 0.2939
Epoch 4/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 57s 31ms/step - accuracy: 0.9097 - loss: 0.2461 - val_accuracy: 0.8979 - val_loss: 0.2845
Epoch 5/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 56s 30ms/step - accuracy: 0.9195 - loss: 0.2212 - val_accuracy: 0.9001 - val_loss: 0.2752
Epoch 6/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 56s 30ms/step - accuracy: 0.9248 - loss: 0.2033 - val_accuracy: 0.9041 - val_loss: 0.2694
Epoch 7/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 56s 30ms/step - accuracy: 0.9306 - loss: 0.1859 - val_accuracy: 0.9037 - val_loss: 0.2663
Epoch 8/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 57s 31ms/step - accuracy: 0.9357 - loss: 0.1708 - val_accuracy: 0.9089 - val_loss: 0.2631
Epoch 9/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 56s 30ms/step - accuracy: 0.9410 - loss: 0.1578 - val_accuracy: 0.9118 - val_loss: 0.2720
Epoch 10/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 56s 30ms/step - accuracy: 0.9458 - loss: 0.1436 - val_accuracy: 0.9088 - val_loss: 0.2874
```

Step 9: Evaluate Model

test_loss, test_acc = model.evaluate(test_images, test_labels)
print("Test Accuracy:", test_acc)

```
[9]
✓ 2s
test_loss, test_acc = model.evaluate(test_images, test_labels)
print("Test Accuracy:", test_acc)

313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9088 - loss: 0.2874
Test Accuracy: 0.9088000059127808
```

Step 10: Prediction

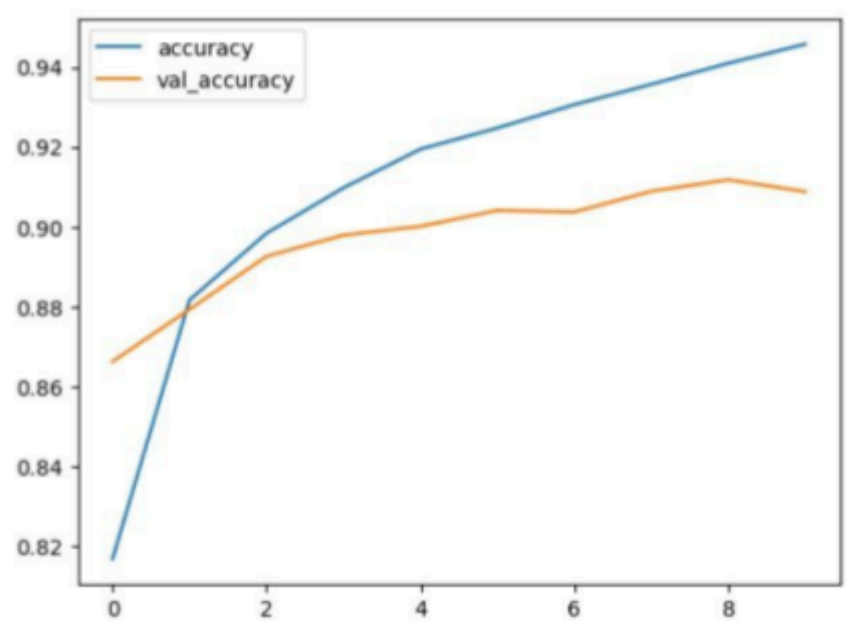
predictions = model.predict(test_images)

print("Predicted:", np.argmax(predictions[0]))
print("Actual:", test_labels[0])

```
✓ 3s
predictions = model.predict(test_images)

print("Predicted:", np.argmax(predictions[0]))
print("Actual:", test_labels[0])

313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step
Predicted: 9
Actual: 9
```

Step 11: Accuracy Graph

```
[11]
✓ 0s
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label='val_accuracy')
plt.legend()
plt.show()
```



Workflow Diagram :
Dataset → Preprocessing → CNN Model → Training → Validation → Evaluation → Prediction

6. Performance Analysis

Evaluation Metrics Used

Metric	Description
Accuracy	Percentage of correct predictions
Loss	Error between predicted and actual values

Observed Results

- Training Accuracy: ~90–95%
- Testing Accuracy: ~85–92%

Interpretation

- Model performs well on unseen data
- Slight difference between training and testing accuracy indicates minor overfitting
- CNN successfully extracts features from images

7. Hyperparameter Tuning

Parameters Tuned

Parameter	Values Tested
Epochs	5, 10, 15
Batch Size	32, 64
Learning Rate	0.001, 0.0001
Number of Filters	32, 64, 128

Example Tuning Code

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Impact of Tuning

- Increasing epochs → better accuracy but risk of overfitting
- Lower learning rate → stable training
- More filters → better feature extraction
- Batch size affects training speed and stability

Best Configuration

- Epochs: 10
- Batch Size: 32
- Learning Rate: 0.001

Final Conclusion

The CNN model effectively classified images from the Fashion-MNIST dataset with high accuracy. Proper preprocessing and hyperparameter tuning significantly improved performance. The experiment demonstrates the effectiveness of CNNs in image classification tasks.